AWS Certified DevOps Engineer – Professional（DOP-C02）　タスク 1.4　#05

コンテナとサーバーレスのデプロイ

ECS のローリング更新（minimumHealthyPercent・maximumPercent）、サーキットブレーカーとデプロイアラーム、ECS のブルー/グリーンとカナリア、CodeDeploy の ECS ブルー/グリーン、imagedefinitions.json と imageDetail.json、EKS、Lambda のバージョン・エイリアス・加重エイリアス、SAM の段階的なデプロイ、API Gateway の Canary リリース

各ページの下に、根拠にした資料（AWS の DOP-C02 試験ガイド、Amazon ECS・CodePipeline・Amazon EKS・AWS Lambda・AWS SAM・API Gateway のガイドの各ページ）を書いた。『』の中は日本語版ドキュメントの原文どおり

第1章

Amazon ECS・EKS のデプロイ

タスクを入れ替えるか、トラフィックを移すか。

ローリング更新と失敗の検知

必要数 4	最小	最大	動き
容量を保つ	100%	200%	先に 4 個起動 → 古い 4 個を止める
半分ずつ	50%	100%	2 個止めて 2 個起動
止めてよい	0%	100%	全部止めて入れ替え（速い）
進まない	100%	100%	止めも増やしもできない

タグはダイジェストに解決（全タスクが同じイメージ）。

サーキットブレーカー
タスクが定常状態にならない → 失敗

デプロイアラーム
CloudWatch アラームが ALARM → 失敗

↓ ロールバックのオプション

COMPLETED の最新のデプロイに戻す
両方を一緒に使うと、どちらかで失敗

出典：タスクを置き換えて Amazon ECS サービスをデプロイする／デプロイサーキットブレーカー／CloudWatch アラームが Amazon ECS のデプロイ障害を検出する方法

トラフィックを移すデプロイ

ALB リスナー（本番 :443 ／ テスト :8443）

↓ ルールで切り替え

ターゲットグループ 1：ブルー
今のリビジョン（タスクセット）

ターゲットグループ 2：グリーン
新しいリビジョン。テストトラフィックで確認

ベイク時間：移した後も両方を動かし、すぐ戻せる。

方法	特徴
ECS ブルー/グリーン	ECS 自身。ライフサイクルフック（Lambda・一時停止）、ベイク時間
ECS カナリア	2 フェーズ。評価期間は両方動く（リソース増）
CodeDeploy	置き換えタスクセット、ターゲットグループ 2 つ、Canary・線形、AfterAllowTestTraffic

NLB では CodeDeploy の ECSAllAtOnce だけ。

出典：Amazon ECS ブルー/グリーンデプロイ／Amazon ECS カナリアデプロイ／Amazon ECS 用の CodeDeploy ブルー/グリーンデプロイ

CodePipeline からのデプロイ

```
# ECS 標準（ローリング）: imagedefinitions.json
[{"name":"web",
  "imageUri":"1111.dkr.ecr.../web:abc123"}]

# CodeDeployToECS（ブルー/グリーン）
imageDetail.json  # {"ImageURI": "..."}
taskdef.json      # "image": "<IMAGE1_NAME>"
appspec.yaml      # TaskDefinition・コンテナ名・ポート
```

アクション	読むファイル	デプロイ
Amazon ECS	imagedefinitions.json	ローリング更新
CodeDeployToECS	imageDetail・taskdef・appspec	ブルー/グリーン
EKS	マニフェスト・Helm	Kubernetes の Deployment

EKS：フルマネージドの Kubernetes。kubectl・Helm をそのまま使う。

出典：Amazon Elastic Container Service デプロイアクションリファレンス／Amazon ECS および CodeDeploy ブルー/グリーンデプロイアクションリファレンス／Amazon EKS とは

第2章

Lambda とサーバーレス

バージョンは固定、エイリアスで切り替える。

バージョン・エイリアス・加重エイリアス

$LATEST
未公開。デプロイのたびに上書き。ここだけ変えられる

↓ 公開（publish）

バージョン 7 ／ バージョン 8
コードと設定が固定

↑ 指す

エイリアス prod（加重：v7 90% ／ v8 10%）
呼び出し側は prod の ARN を使う

新しいバージョン＝エイリアスの向き先を変えるだけ。戻すのも同じ。

公開しても prod が指していなければ本番に影響なし（ベータ用）。

$LATEST が前のバージョンと同じなら新しいバージョンは作れない。

トラフィックが少ないと、加重の割合と実際がずれる。

出典：Lambda 関数のバージョンを管理する／Lambda 関数のエイリアスの作成／加重エイリアスを使用して Lambda カナリアデプロイの実装

SAM の段階的なデプロイと API Gateway

```
Transform: AWS::Serverless-2016-10-31
Resources:
  Api:
    Type: AWS::Serverless::Function
    Properties:
      AutoPublishAlias: live # 公開＋エイリアス
      DeploymentPreference:
        Type: Canary10Percent10Minutes
        Alarms: [ !Ref ErrorsAlarm ]
        Hooks:
          PreTraffic: !Ref PreFn  # 移す前
          PostTraffic: !Ref PostFn # 移した後
```

CodeDeploy のリソースは SAM が自動で作る。

テスト失敗・アラーム発火
CodeDeploy がロールバック

API Gateway の Canary リリース
ステージのトラフィックを比率でランダムに分ける

よければ昇格して Canary を無効に
だめなら Canary を外す

出典：AWS SAM とは／を使用してサーバーレスアプリケーションを徐々にデプロイする AWS SAM／API Gateway の Canary リリースデプロイの設定

本番での問われ方

ねらい	見るところ
ECS ローリング	最小・最大の割合の計算、サーキットブレーカー（起動失敗）とアラーム（メトリクス）、ロールバック
ECS トラフィック移行	ブルー/グリーンとベイク時間、カナリアの評価期間、CodeDeploy のターゲットグループ 2 つ・テストリスナー
パイプライン	imagedefinitions.json（標準）と imageDetail.json・taskdef.json・appspec.yaml（ブルー/グリーン）
Lambda	$LATEST・バージョン・エイリアス・加重エイリアス、SAM の AutoPublishAlias・DeploymentPreference・PreTraffic／PostTraffic・Alarms
API	API Gateway の Canary リリースと昇格

出典：DOP-C02 試験ガイド タスク 1.4／本資料の問題演習（全 30 問）